

Curso online: **Programación orientada a objetos: Java avanzado**

Servicios web

Autores

Jorge Molinero Muñoz

Miguel Sallent Sánchez

ÍNDICE

1. Introducción a los servicios web.....	3
2. Creación de un servicio web con JAX-RS.....	4
3. Consumir el servicio web a través de SoapUI.....	15
4. Consumir el servicio desde un cliente.....	21

1. Introducción a los servicios web

Un Servicio Web es un servicio al que podemos acceder mediante protocolos Web estándar como HTTP, utilizando lenguajes como XML o JSON para el intercambio de información.

Los Servicios Web son un mecanismo que permite exponer la funcionalidad de nuestro sistema para que sea **consumido por otras aplicaciones**, de modo que se publique una colección de procedimientos (o métodos) a los que podemos llamar desde cualquier lugar de Internet, proporcionando independencia de la plataforma que estemos utilizando y del lenguaje de programación en el que se haya implementado internamente el servicio.

Por ejemplo, podríamos tener un sistema de información para el Registro Electrónico de una administración pública, y deseamos que todas las aplicaciones de esa administración puedan utilizar el registro, consultando entradas y generando salidas. En ese caso creamos un servicio web que expone la funcionalidad del registro. Las aplicaciones que quieran utilizar el registro tendrán que implementar un cliente que se conecte al servicio para realizar las acciones.

Por tanto cuando hablamos de servicios web siempre tendremos las siguientes aplicaciones:

- un servidor (el servicio web)
- uno o varios clientes (consumidores del servicio web)

La ventaja de los servicios web es que cliente y servidor quedan desacoplados, de forma que pueden ser desarrollados con tecnologías distintas. Por ejemplo el servidor puede estar desarrollado con tecnología Java y el cliente con tecnología .Net. O varios clientes cada uno con una tecnología diferente. En nuestro trabajo puede que necesitemos desarrollar sólo el servidor (si queremos publicar un servicio a terceros), sólo el cliente (si queremos consumir un servicio de terceros), o ambos.

En la administración pública hay muchos ejemplos de aplicaciones que publican servicios web, principalmente para servicios comunes, como pueden ser registro electrónico, portafirmas, envío de mensajes SMS, validación de certificados (@firma), plataforma de intermediación de datos, base de datos nacional de subvenciones, etc.

Componentes de los Servicios Web

Los servicios web utilizan **protocolos de transporte** estándar como HTTP o HTTPS, lo que permite a cliente y servidor estar alojados en máquinas distintas (el único requisito es que tengan conectividad, bien a través de Internet o mediante una red privada).

Los mensajes se codifican utilizando **lenguajes** como XML o JSON, lenguajes de uso generalizado que permiten una rápida integración.

Finalmente se define un **protocolo** que define cómo intercambiar los mensajes, como puede ser SOAP o REST.

Tipos de servicios web.

Tradicionalmente el protocolo empleado en la comunicación con los servicios web ha sido SOAP: *Simple Access Object Protocol* o protocolo de acceso simple a objetos. Este protocolo está basado fuertemente en el uso de XML, el cliente y el servidor intercambiaban información del servicio web usando mensajes estructurados en XML.

El lenguaje XML ha sido tan popular en el intercambio de información, que hasta los navegadores web se adaptaron a dicho lenguaje, permitiendo la comunicación por medio de AJAX (*Asynchronous Javascript and XML*) con el servidor web. Esta tecnología permitía al navegador realizar peticiones con javascript a los servicios web expuestos en el servidor, e interpretar los mensajes XML de respuesta.

Sin embargo, SOAP no hace honor a su nombre ("Simple" Object Access Protocol), y la comunicación implica una enorme cantidad de etiquetas XML para describir el mensaje, las acciones y el servicio web. El resultado es que el desarrollo de servicios web SOAP o sus clientes a día de hoy sigue siendo teniendo una complejidad media o alta dependiendo de la tecnología que se use, incluso aunque se utilice alguna de las herramientas de generación automática de código disponibles.

Posteriormente apareció un nuevo protocolo, el protocolo REST (*Representational State Transfer*).

La clave del funcionamiento de REST es que está fuertemente basado en el protocolo HTTP, el mismo que se usa para el funcionamiento de la web en general. En REST, el elemento principal son los recursos, que son identificados por una URL y las acciones que se pueden realizar sobre ellos se indican a través de los distintos métodos http de petición: GET, POST, PUT, PATCH y DELETE.

Por otro lado, la representación del recurso no está casada con ninguna tecnología, se puede usar XML, HTML, o uno de los formatos más populares actualmente: JSON.

La principal ventaja de los servicios REST es su sencillez, lo que permite desarrollos rápidos en cualquier tecnología.

A este tipo de servicios web se les suele denominar API.

En este curso veremos cómo desarrollar con Java un servicio web de tipo REST, debido a su mayor sencillez frente a SOAP y a que hoy en día es la tendencia a la hora de desarrollar nuevos servicios.

Nota: También es posible desarrollar servicios web SOAP, utilizando frameworks como [CXF](#).

2. Creación de un servicio web con JAX-RS

La especificación Java EE incluye **JAX-RS**, que es una forma estándar de desarrollar servicios Web REST en la plataforma JAVA. La especificación define una serie de anotaciones que simplifican el desarrollo y despliegue de los servicios.

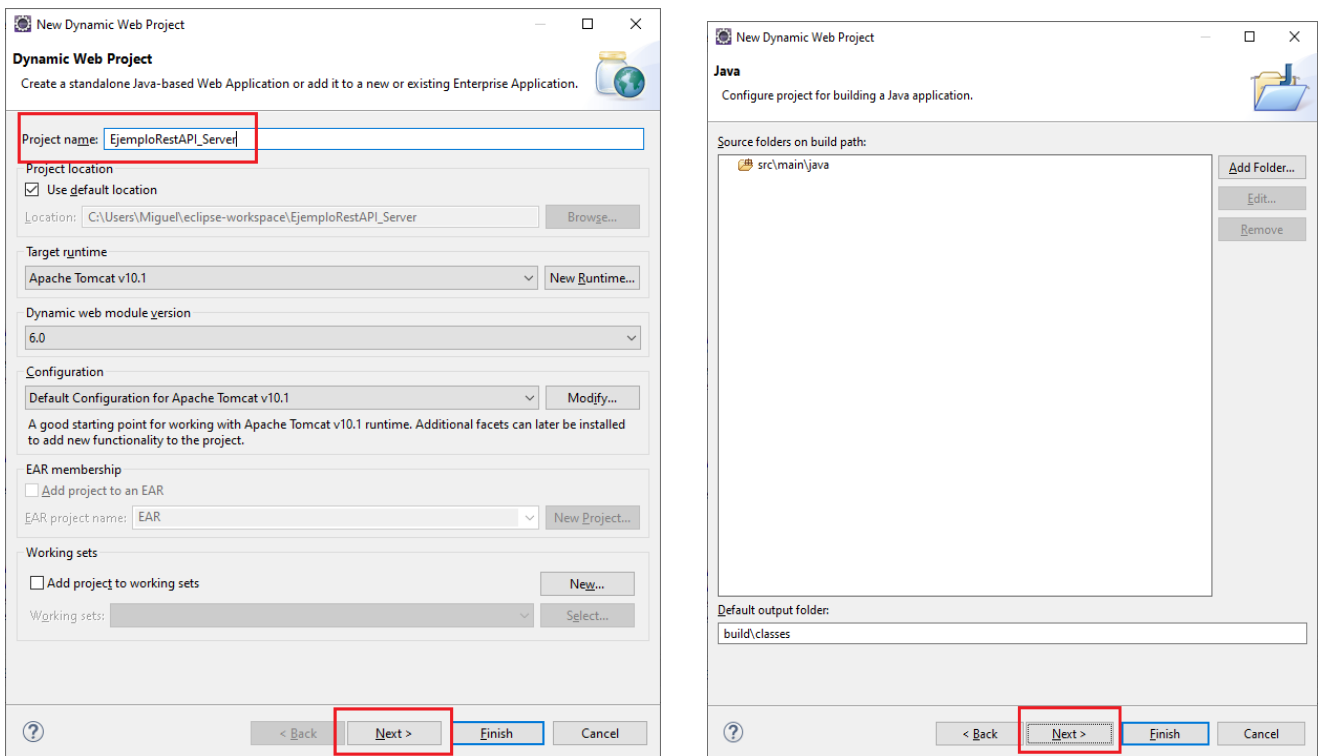
JAX-RS es un API de alto nivel, lo cual quiere decir que únicamente se define la sintaxis, pero no la implementación concreta, que no se incluye en la distribución J2EE

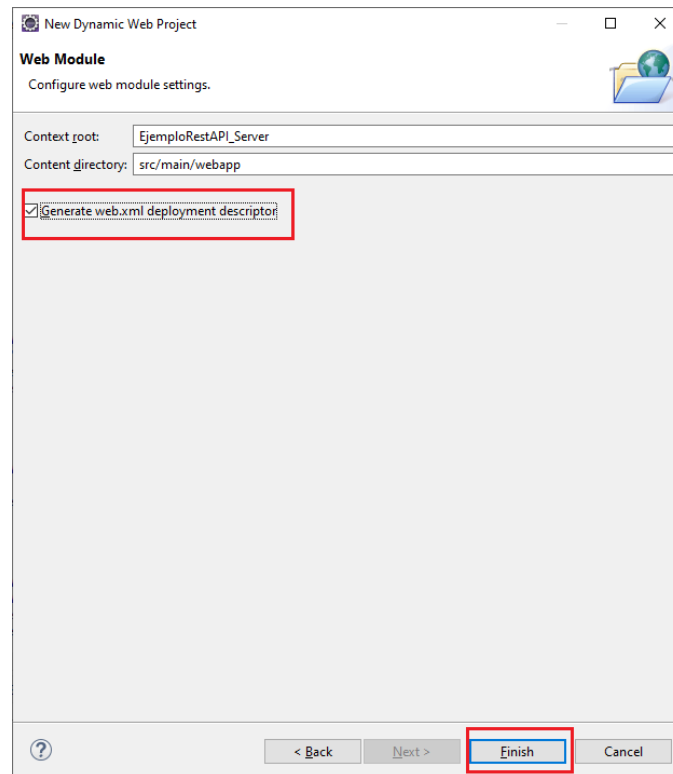
proporcionada por Oracle y debe ser añadida de forma externa. En nuestro caso vamos a utilizar un framework denominado *Jersey*.

Además utilizaremos el gestor de paquetes *Maven*, que nos permite importar las librerías externas de una manera rápida y sencilla.

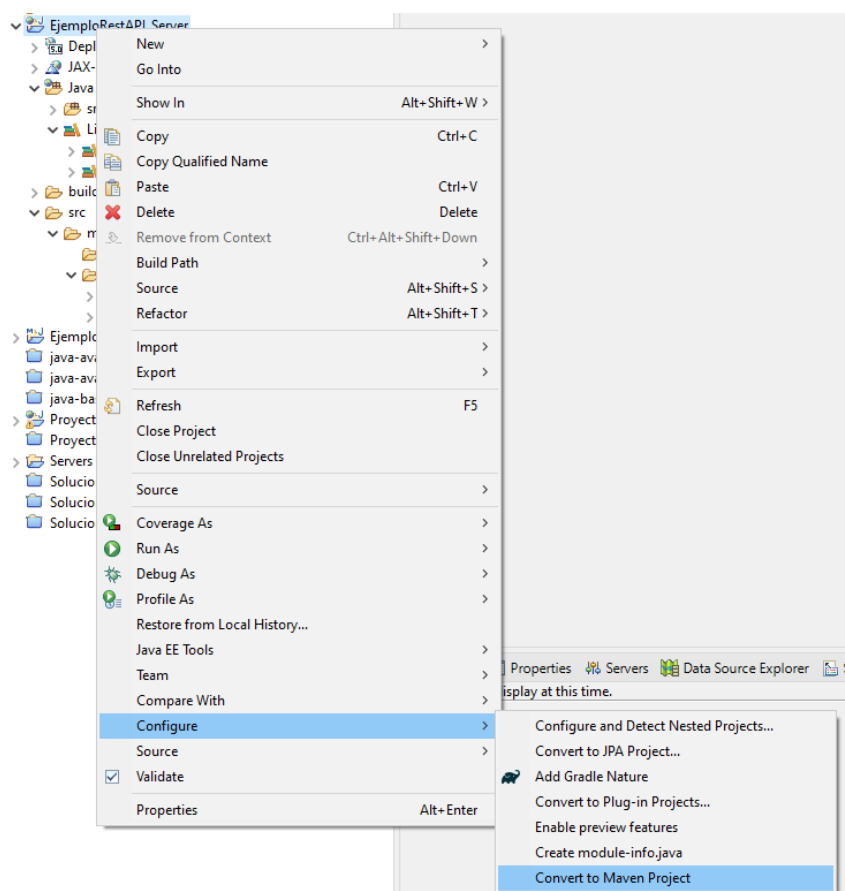
Mediante el siguiente ejemplo veremos cómo desarrollar el servicio.

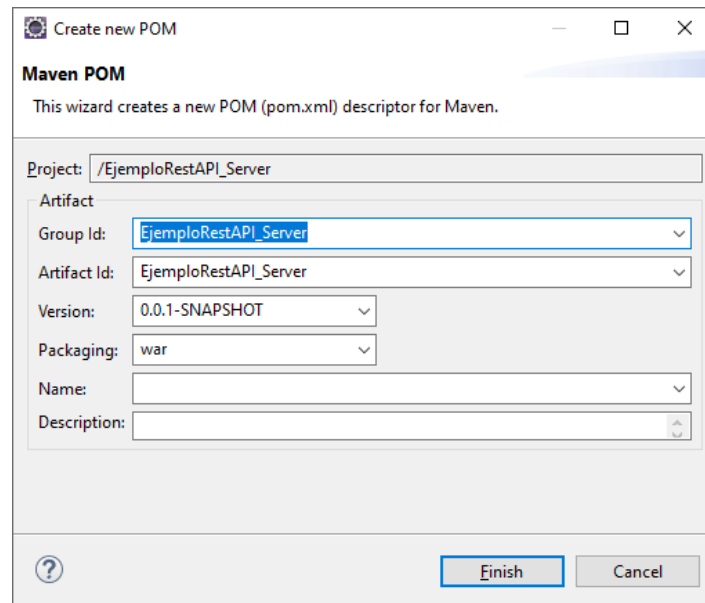
En primer lugar crearemos un proyecto, de tipo *Dynamic Web Project*:



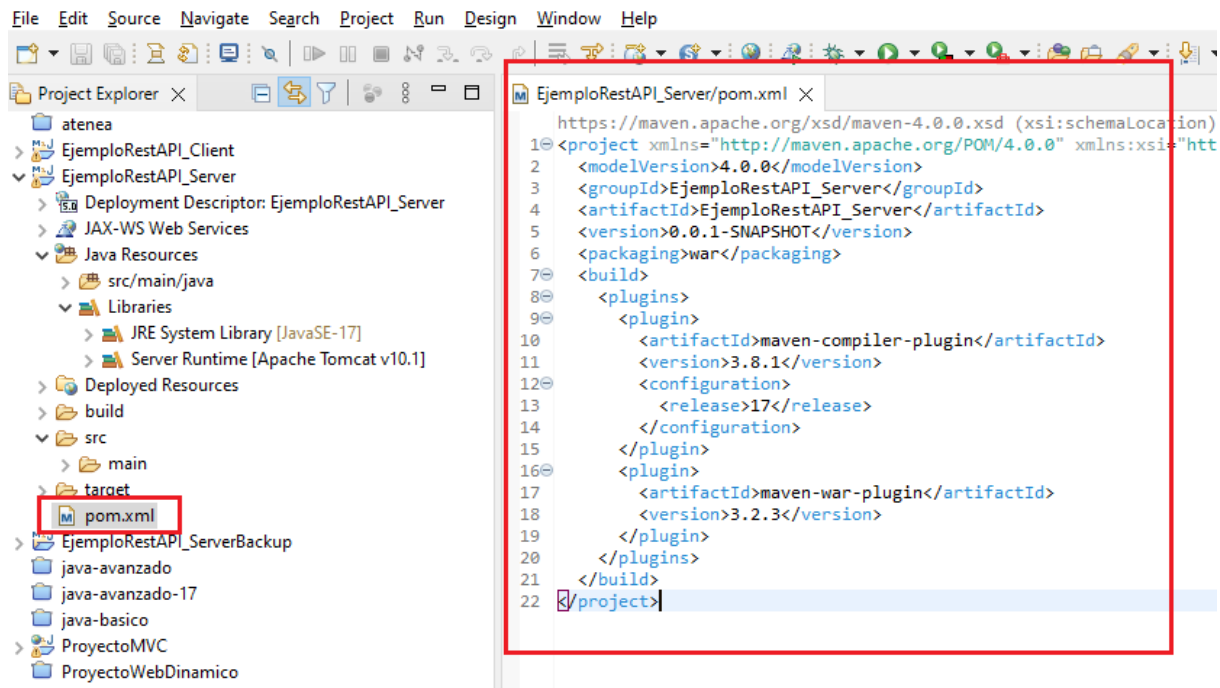


A continuación importaremos las librerías necesarias mediante Maven. Para ello en primer lugar pulsaremos con el botón derecho sobre el proyecto y seleccionaremos *Configure → Convert to Maven Project*. Dejamos las opciones por defecto:





Observamos que se ha creado un fichero *pom.xml* a nivel raíz del proyecto:



Este archivo xml tiene una sección *build*. Nosotros añadiremos delante una sección *dependencies* con todas las dependencias que necesitamos:

```
<dependencies>
  <dependency>
    <groupId>jakarta.ws.rs</groupId>
    <artifactId>jakarta.ws.rs-api</artifactId>
    <version>4.0.0</version>
  </dependency>
  <dependency>
    <groupId>org.glassfish.jersey.containers</groupId>
```

```

        <artifactId>jersey-container-servlet</artifactId>
        <version>3.1.7</version>
    </dependency>
    <dependency>
        <groupId>org.glassfish.jersey.inject</groupId>
        <artifactId>jersey-hk2</artifactId>
        <version>3.1.7</version>
    </dependency>
    <dependency>
        <groupId>org.glassfish.jersey.media</groupId>
        <artifactId>jersey-media-json-jackson</artifactId>
        <version>3.1.7</version>
    </dependency>

    <!-- para la generación del wadl -->
    <dependency>
        <groupId>org.glassfish.jaxb</groupId>
        <artifactId>jaxb-runtime</artifactId>
        <version>4.0.5</version>
    </dependency>
</dependencies>

```



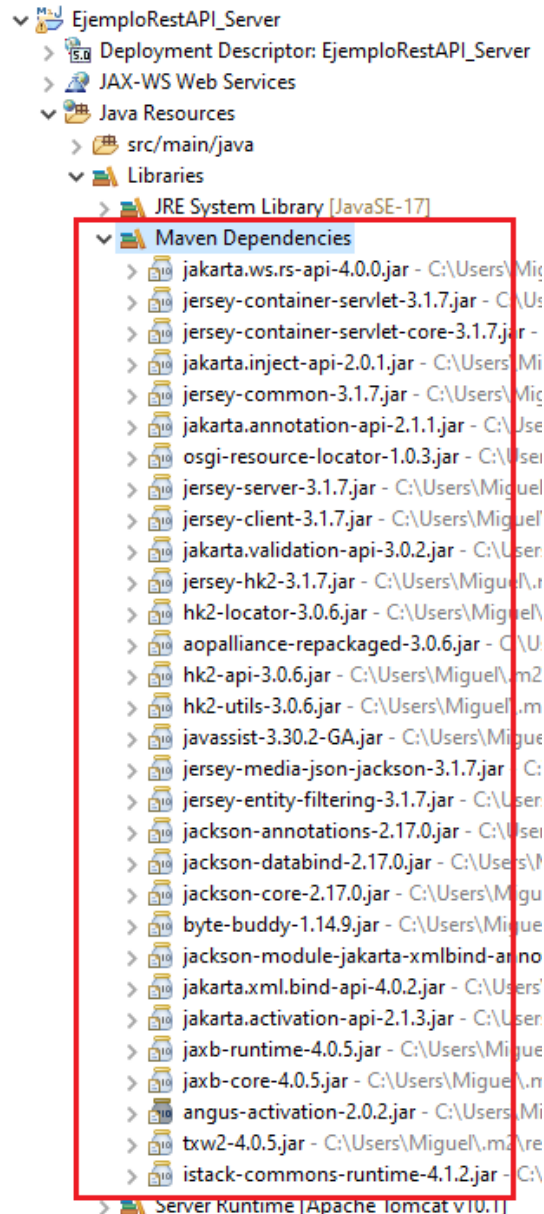
The screenshot shows a code editor with a file named 'EjemploRestAPI_Server/pom.xml'. The XML content is as follows:

```

1  <groupId>org.glassfish.jersey.samples</groupId>
2  <artifactId>EjemploRestAPI_Server</artifactId>
3  <version>0.0.1-SNAPSHOT</version>
4  <packaging>war</packaging>
5  <dependencies>
6      <dependency>
7          <groupId>jakarta.ws.rs</groupId>
8          <artifactId>jakarta.ws.rs-api</artifactId>
9          <version>4.0.0</version>
10     </dependency>
11     <dependency>
12         <groupId>org.glassfish.jersey.containers</groupId>
13         <artifactId>jersey-container-servlet</artifactId>
14         <version>3.1.7</version>
15     </dependency>
16     <dependency>
17         <groupId>org.glassfish.jersey.inject</groupId>
18         <artifactId>jersey-hk2</artifactId>
19         <version>3.1.7</version>
20     </dependency>
21     <dependency>
22         <groupId>org.glassfish.jersey.media</groupId>
23         <artifactId>jersey-media-json-jackson</artifactId>
24         <version>3.1.7</version>
25     </dependency>
26
27     <!-- para la generación del wadl -->
28     <dependency>
29         <groupId>org.glassfish.jaxb</groupId>
30         <artifactId>jaxb-runtime</artifactId>
31         <version>4.0.5</version>
32     </dependency>
33 </dependencies>
34 <build>
35     <plugins>

```


Maven lo que hace es descargar automáticamente de sus repositorios las librerías indicadas. Podemos observar que aparece una nueva sección *Maven Dependencies* en *Java Resources*, con las librerías indicadas descargadas e incorporadas al proyecto. La gran ventaja de Maven es que cada paquete a su vez indica sus dependencias (con su versión), por lo que se descargan también de forma recursiva, y nos abstrae del proceso de gestionar todas las interdependencias. Así, observamos que aunque hemos indicado sólo 5 paquetes, se nos han descargado muchos más:



Advertencia: para que Maven funcione, es necesario tener conectividad a través de Internet con los repositorios de Maven. Si estamos dentro de la red del trabajo es posible que esta conectividad esté limitada y la descarga no funcione por este motivo. Para este caso se facilitan todas las librerías en la web del curso, el alumno en esta situación deberá descargarlas e incorporarlas al proyecto copiándolas en la carpeta WEB-INF/lib del proyecto.

A continuación crearemos una clase Ciudadano que representará el objeto que queremos manipular, con atributos *id*, *nombre* y *apellidos*, en el paquete *restserver.data*:

```

package restserver.data;

public class Ciudadano {
    private int id;
    private String nombre;
    private String apellidos;

    public int getId() {
        return id;
    }

    public void setId(int id) {
        this.id = id;
    }

    public String getNombre() {
        return nombre;
    }

    public void setNombre(String nombre) {
        this.nombre = nombre;
    }

    public String getApellidos() {
        return apellidos;
    }

    public void setApellidos(String apellidos) {
        this.apellidos = apellidos;
    }
}

```

Para evitar la incorporación de una base de datos, simularemos la persistencia mediante una clase CiudadanosDAO:

```

package restserver.dao;

import java.util.ArrayList;
import java.util.Collections;
import java.util.Optional;

import restserver.data.Ciudadano;

public class CiudadanosDAO {
    private ArrayList<Ciudadano> ciudadanos = new ArrayList<Ciudadano>();

    private static CiudadanosDAO instance = new CiudadanosDAO();

    static {
        Ciudadano c = new Ciudadano();
        c.setId(1);
        c.setNombre("Nombre 1");
        c.setApellidos("Apellidos 1");

        instance.ciudadanos.add(c);

        c = new Ciudadano();
    }
}

```

```

        c.setId(2);
        c.setNombre("Nombre 2");
        c.setApellidos("Apellidos 2");

        instance.ciudadanos.add(c);
    }

    public static CiudadanosDAO getInstance() {
        return instance;
    }

    public Ciudadano getCiudadanoById(int id) {
        Ciudadano result = null;
        Optional<Ciudadano> ciudadano = ciudadanos.stream().filter(c -> c.getId() == id).findFirst();
        if (ciudadano.isPresent()) {
            result = ciudadano.get();
        }
        return result;
    }

    public ArrayList<Ciudadano> getCiudadanos() {
        return ciudadanos;
    }

    public void updateCiudadano(Ciudadano ciudadano) {
        if (ciudadano.getId() == 0) {
            // insertar
            ciudadano.setId(Collections.max(ciudadanos.stream().map(c -> c.getId()).toList()) + 1);
            ciudadanos.add(ciudadano);
        } else {
            // actualizar
            for (int i = 0; i < ciudadanos.size(); i++) {
                if (ciudadanos.get(i).getId() == ciudadano.getId()) {
                    ciudadanos.set(i, ciudadano);
                }
            }
        }
    }
}

```

Hasta ahora las clases que tenemos no se diferencian de las que habríamos hecho para una aplicación web clásica. A continuación desarrollaremos las clases que implementan propiamente el servicio web.

Crearemos un paquete *restserver.api* con dos clases:

CiudadanoAPI.java: clase que implementa el servicio en sí:

```

package restserver.api;

import jakarta.ws.rs.Consumes;
import jakarta.ws.rs.GET;
import jakarta.ws.rs.POST;
import jakarta.ws.rs.Path;

```

```

import jakarta.ws.rs.PathParam;
import jakarta.ws.rs.Produces;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.core.Response;
import restserver.dao.CiudadanosDAO;
import restserver.data.Ciudadano;

@Path("ciudadano")
public class CiudadanoAPI {

    @GET
    @Path("/{id}")
    @Produces(MediaType.APPLICATION_JSON)
    public Response getById(@PathParam("id") int id) {
        Ciudadano ciudadano = CiudadanosDAO.getInstance().getCiudadanoById(id);
        return Response.ok().entity(ciudadano).build();
    }

    @GET
    @Produces(MediaType.APPLICATION_JSON)
    public Response get() {
        return Response.ok().entity(CiudadanosDAO.getInstance().getCiudadanos()).build();
    }

    @POST
    @Produces(MediaType.APPLICATION_JSON)
    @Consumes(MediaType.APPLICATION_JSON)
    public Response edit(Ciudadano ciudadano) {
        CiudadanosDAO.getInstance().updateCiudadano(ciudadano);
        return Response.ok().build();
    }
}

```

El primer método devuelve el ciudadano con el id indicado. El segundo método devuelve todos los ciudadanos. Y el tercero actualiza un ciudadano.

En esta clase tenemos que prestar atención a las anotaciones.

Observamos que los dos primeros métodos se invocan mediante el método GET de http (anotación @GET), mientras que el tercero se invoca mediante el método POST (anotación @POST).

La anotación @Path para la clase tiene el valor 'ciudadano', lo cual quiere decir que todos los métodos estarán a partir de la url /ciudadano/. Después cada método puede opcionalmente incorporar la anotación @Path si se desea que se invoque en un path distinto. En nuestro caso no será necesario, salvo el primer método, ya que indicaremos el id en el propio path (por ejemplo con la url /ciudadano/1). Para ello usamos la anotación @Path en el método y además la anotación @PathParam en el argumento.

Es decir, los métodos segundo y tercero se invocarán en la misma url y los distinguiremos porque uno se invocará con el método http get y el otro con el método http post. También sería posible configurar cada uno en una url distinta pero no es necesario.

También debemos indicar en qué formato queremos que devuelva el resultado (JSON), con la anotación `@Produces`. Podríamos usar XML si lo preferimos. El tercer método, al utilizar POST, también debe definir el formato de los datos de entrada, mediante la anotación `@Consumes`.

Como vemos, a través de anotaciones hemos definido completamente la interfaz del API, esto es, las urls de los métodos y el formato de los argumentos y de los resultados.

Para arrancar el servicio necesitamos otra clase, que llamaremos `App.java`, y que lo único que hace es exponer la url donde se publicará el API:

```
package restserver.api;

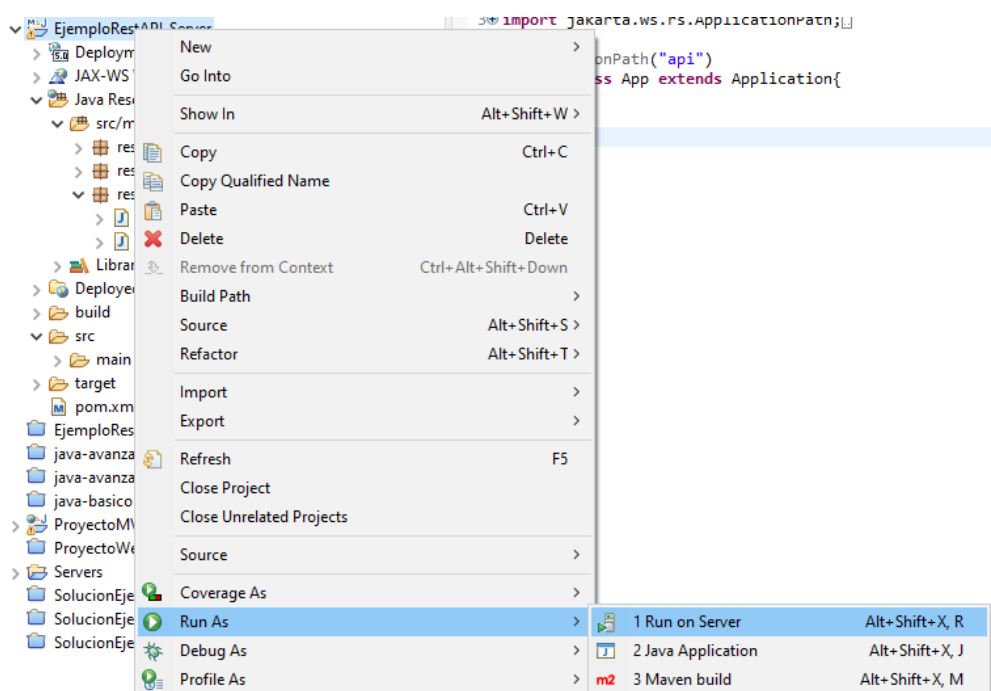
import jakarta.ws.rs.ApplicationPath;
import jakarta.ws.rs.core.Application;

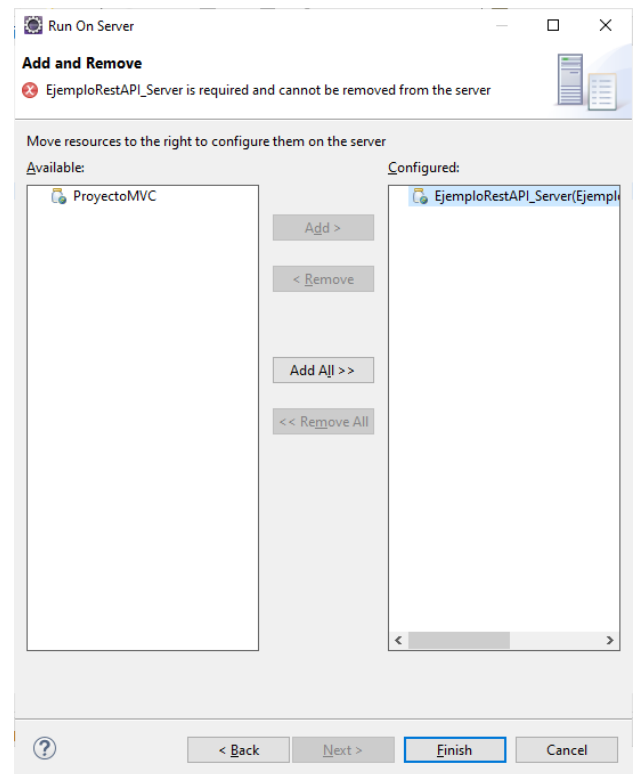
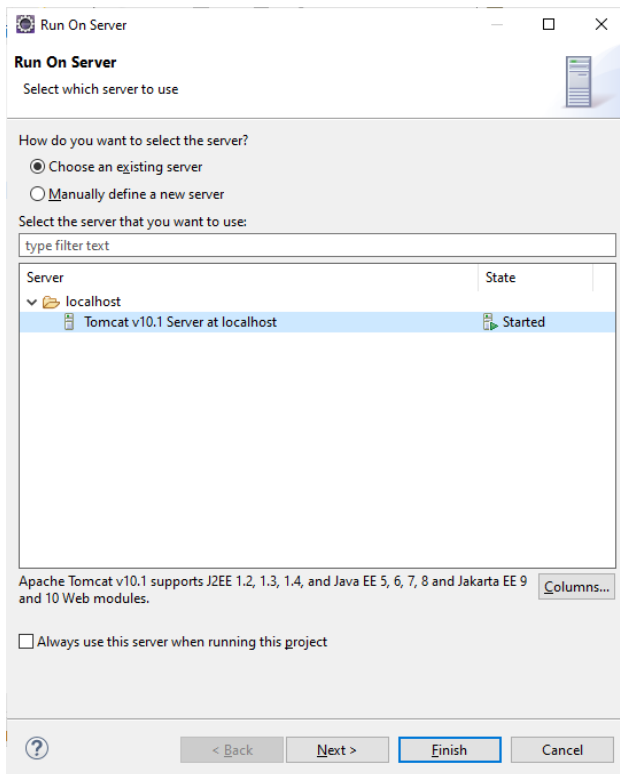
@ApplicationPath("api")
public class App extends Application{

}
```

Mediante la anotación `@ApplicationPath` indicamos la ruta donde estará publicado el servicio. Por ejemplo, para obtener el ciudadano con `id = 1` utilizaríamos la url `/api/ciudadano/1`.

Para probar el desarrollo, ejecutamos la aplicación en Tomcat mediante la opción *Run on Server* (botón derecho del ratón sobre el proyecto):

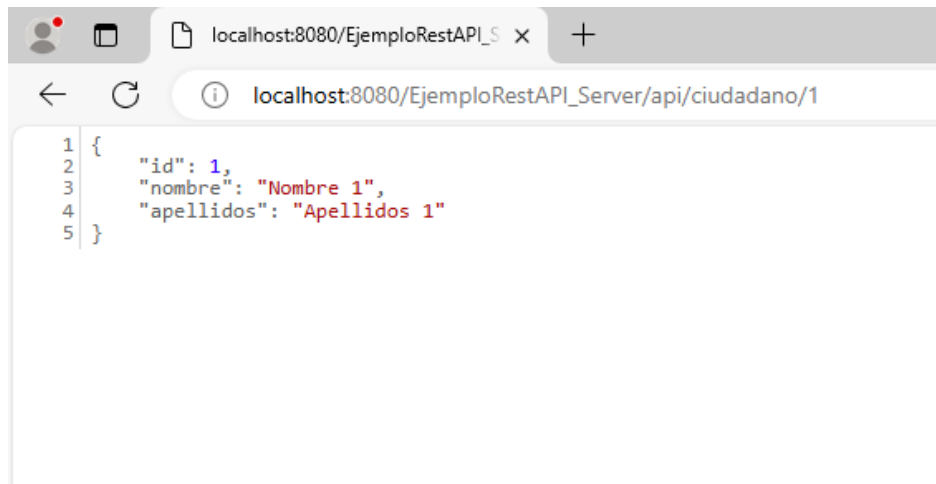




Se abrirá un navegador que mostrará un 404 (not found) ya que a nivel raíz no hemos publicado ningún recurso. Es normal.

Por último probaremos en el navegador las urls siguientes, comprobando que funcionan correctamente los métodos primero y segundo:





Observamos que el formato devuelto es JSON, que es el que habíamos indicado.

Como vemos, para añadir un REST API a nuestra aplicación web únicamente hemos necesitado importar las librerías con Maven y desarrollar dos clases, de una forma muy sencilla.

Hay que resaltar que, aunque hemos consumido el servicio desde un navegador, esto es sólo para probar que el servicio funciona correctamente, ya que lo normal es que el servicio se consuma desde otra aplicación, como veremos en los siguientes apartados.

Finalmente, para probar el método *edit* necesitamos enviar un POST. Se podría probar desde el navegador creando una página html que implementara un formulario para ejecutar el POST, sin embargo hay herramientas como SoapUI que nos facilitan esta tarea, y que veremos a continuación.

3. Consumir el servicio web a través de SoapUI

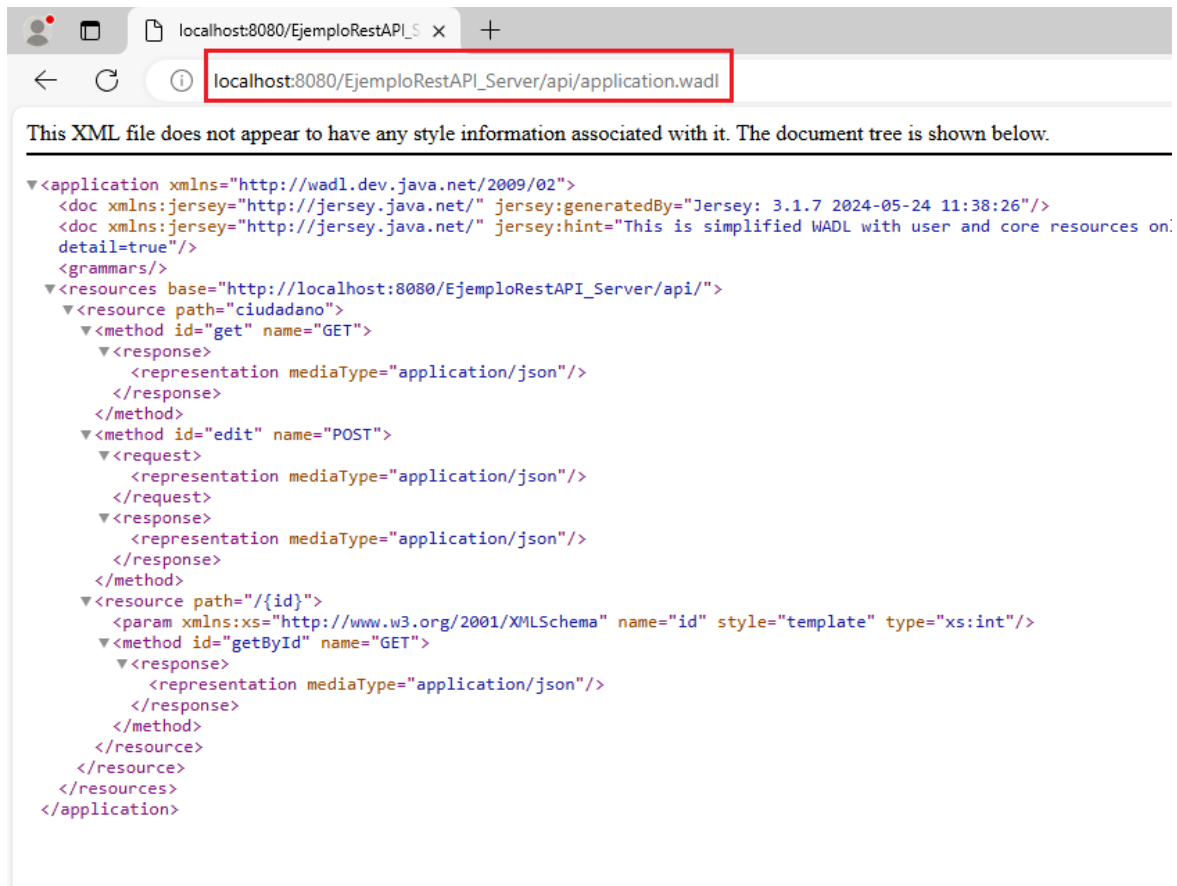
SoapUI es una herramienta que permite probar servicios web, tanto SOAP como REST. Es una herramienta de código abierto y completamente gratuita.

<https://www.soapui.org/>

La instalación de SoapUI es muy sencilla, basta con seguir el típico asistente de instalación. La herramienta que se debe instalar es “SoapUI Open Source”.

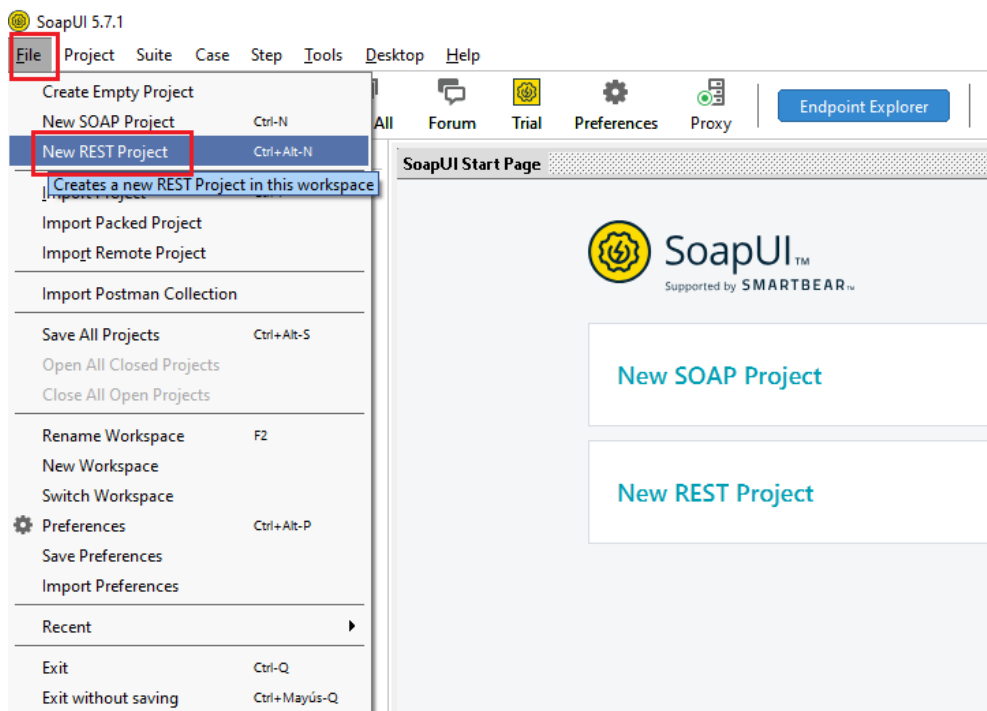
Antes de continuar hay que mencionar que, al haber incluido la librería *jaxb-runtime* en el proyecto, automáticamente en nuestra aplicación se ha generado una url “*application.wadl*” que describe el servicio, y que nos será útil a la hora de configurar SoapUI (es necesario tener el servicio web ejecutándose, como se ha explicado en el apartado anterior):

http://localhost:8080/EjemploRestAPI_Server/api/application.wadl

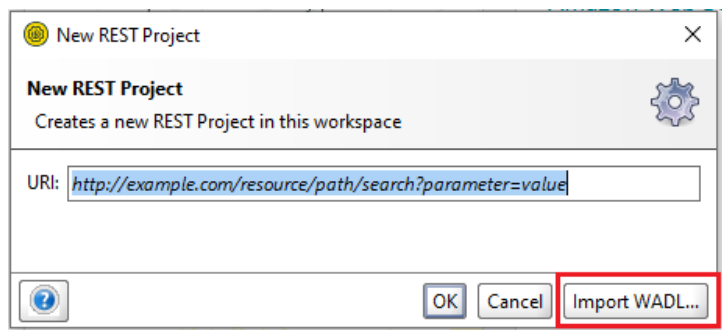


Este archivo WADL describe los diferentes métodos del API de una manera estándar. Sería el equivalente al archivo WSDL en los servicios web SOAP.

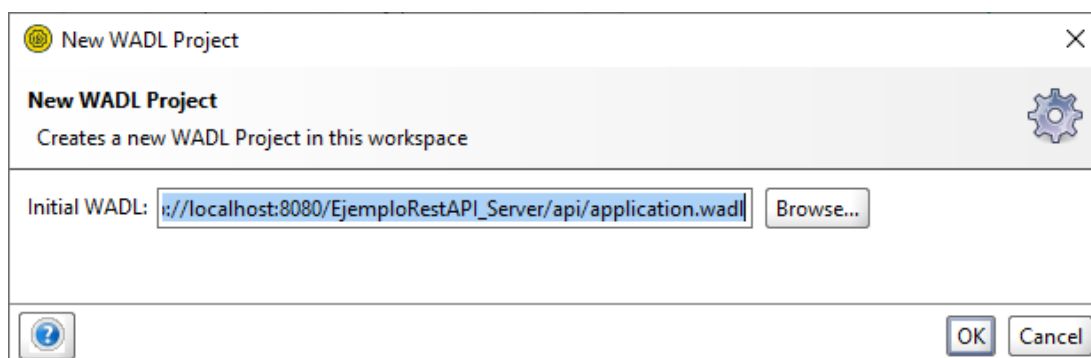
Una vez comprobemos que accedemos al WADL, abriremos SoapUI y crearemos un proyecto REST:



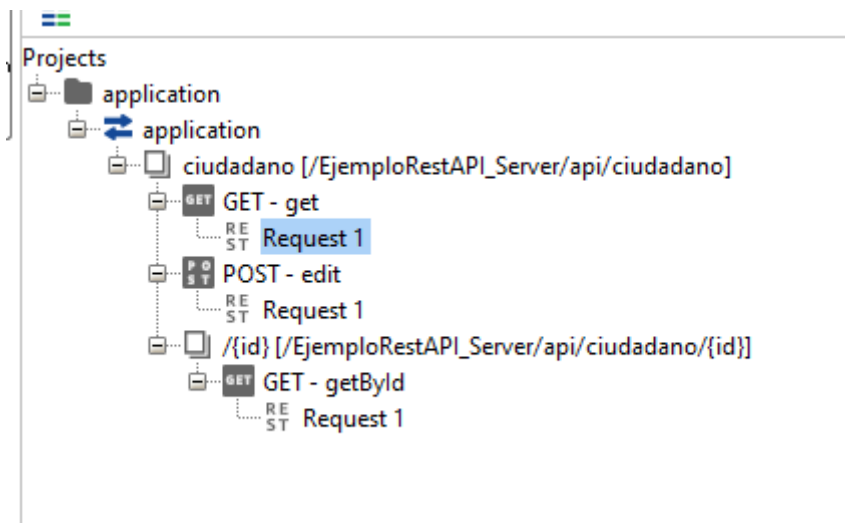
Pulsamos el botón “Import WADL”:



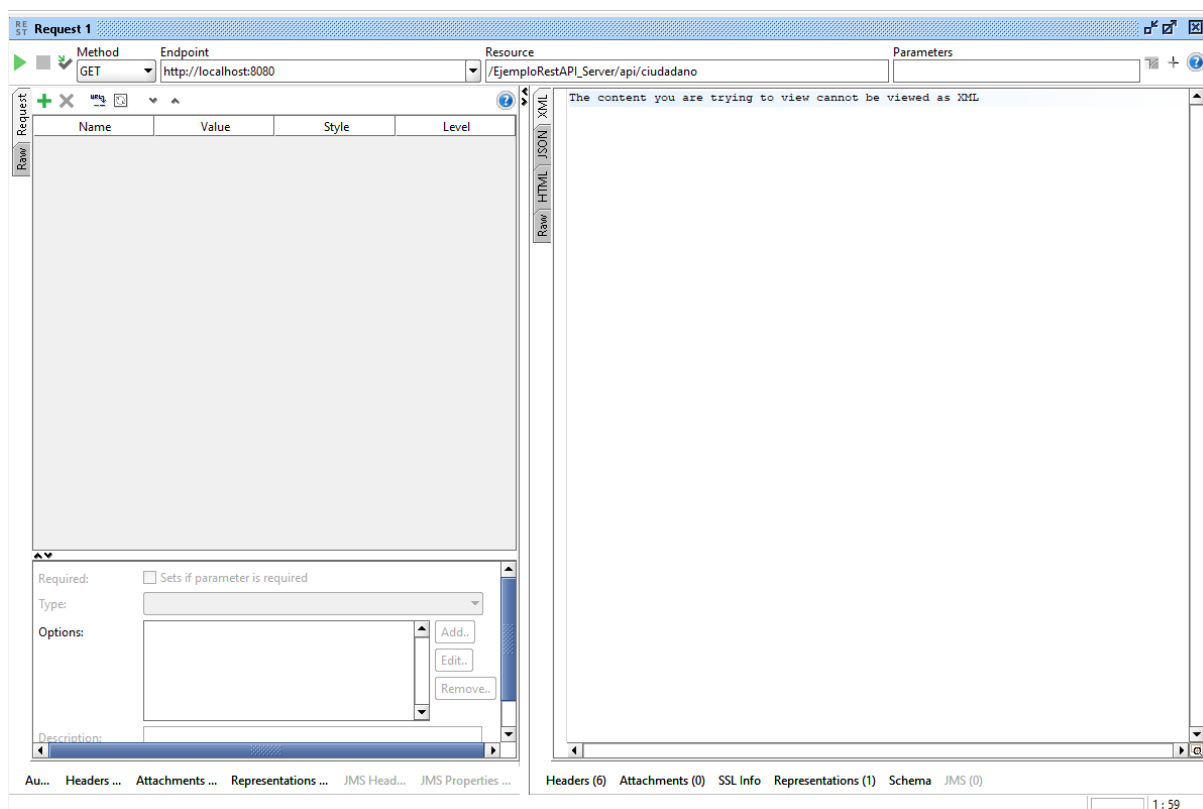
Indicamos la URL del WADL:



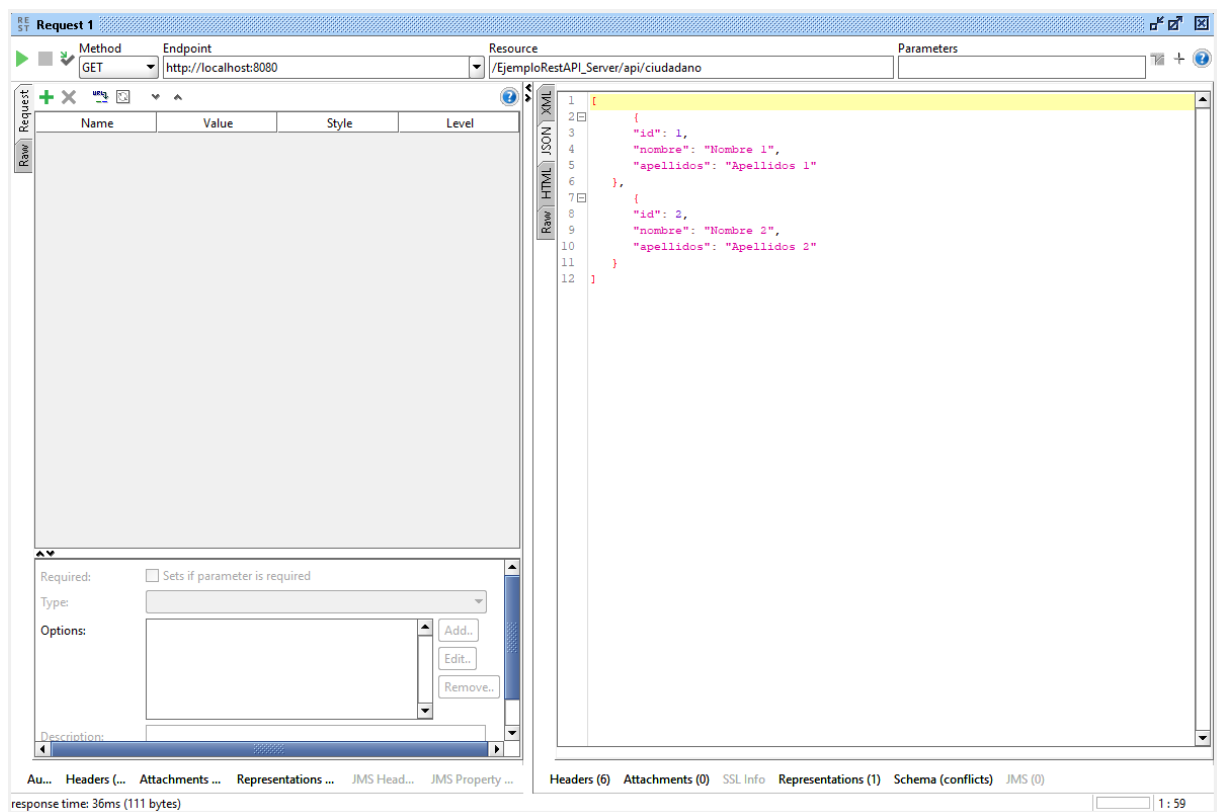
Automáticamente creará el proyecto con los métodos principales expuestos por el servicio web.



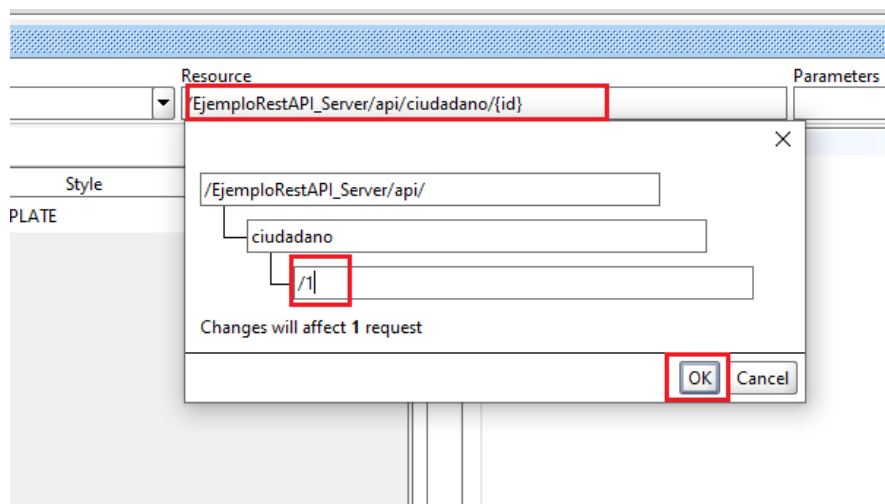
El proyecto automáticamente creará para cada uno de los métodos una petición “Request1”, de modo que al hacer doble click se abre una ventana que se emplearía para invocar al método correspondiente. Si pulsamos sobre la Request asociada al método get veremos la siguiente pantalla:



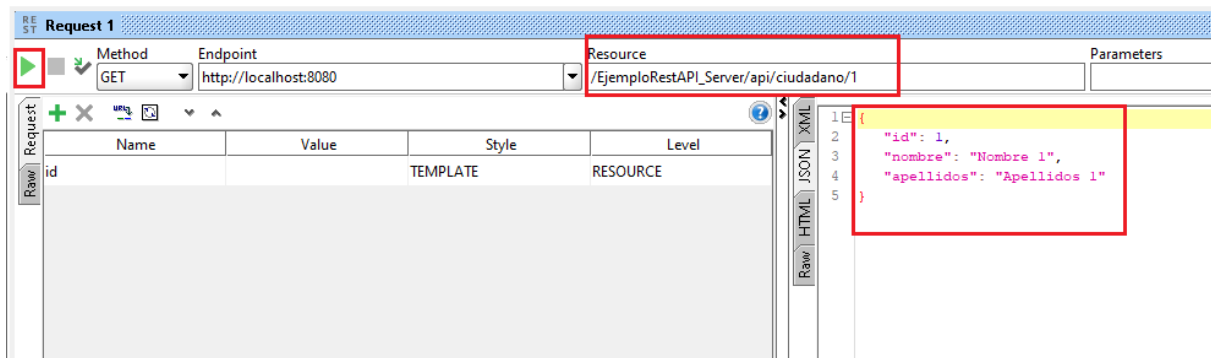
Simplemente pulsando el botón  ejecutaremos la petición, viendo el mensaje de respuesta:



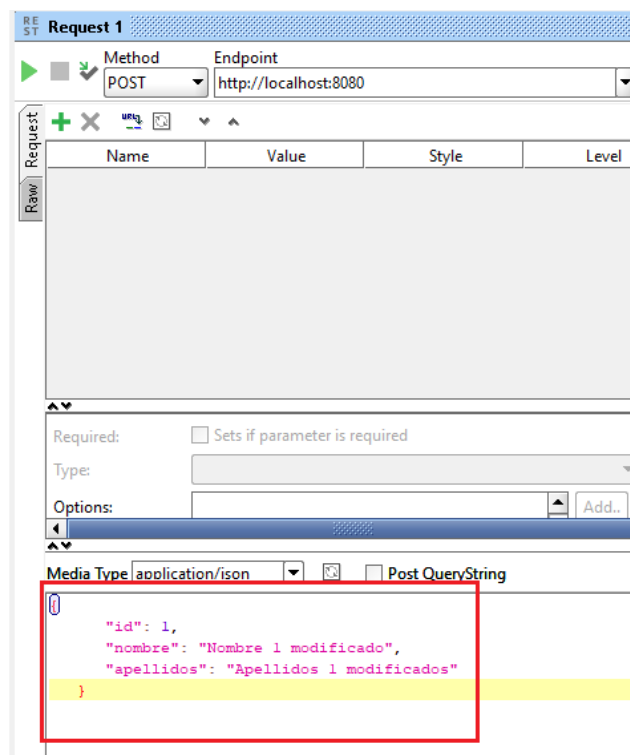
Para probar el método get con id, abriremos su request correspondiente y añadiremos el argumento id pulsando sobre el campo *Resource* y editándolo:



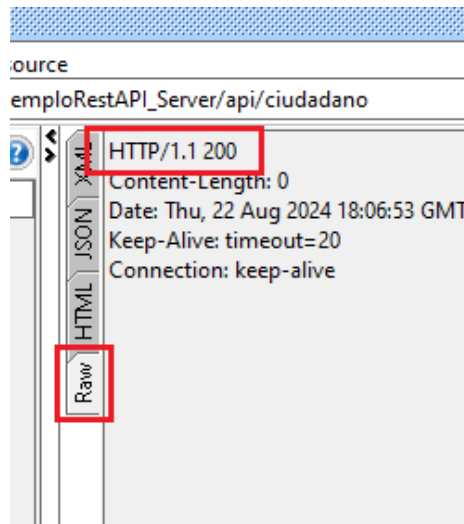
Al ejecutar observamos el resultado:



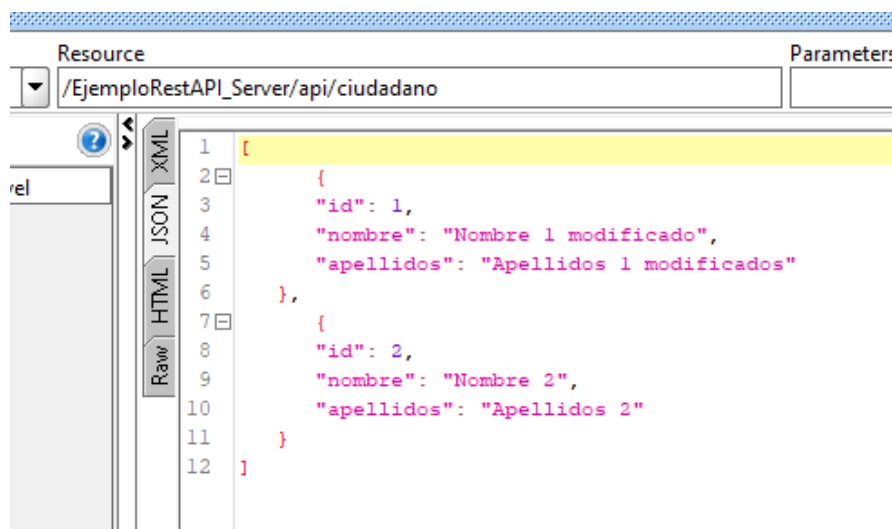
El método *edit* es el que utiliza POST, recordemos que es el motivo principal de usar SoapUI. Para probarlo abrimos su request y rellenamos de la siguiente manera:



Al ejecutar vemos que aparentemente no hace nada, ya que este método no devuelve valor. Sin embargo si observamos el resultado en la pestaña "Raw", vemos que ha devuelto el código HTTP 200 que equivale a OK:



Si ahora ejecutamos de nuevo el primer método get observaremos que se han aplicado los cambios:

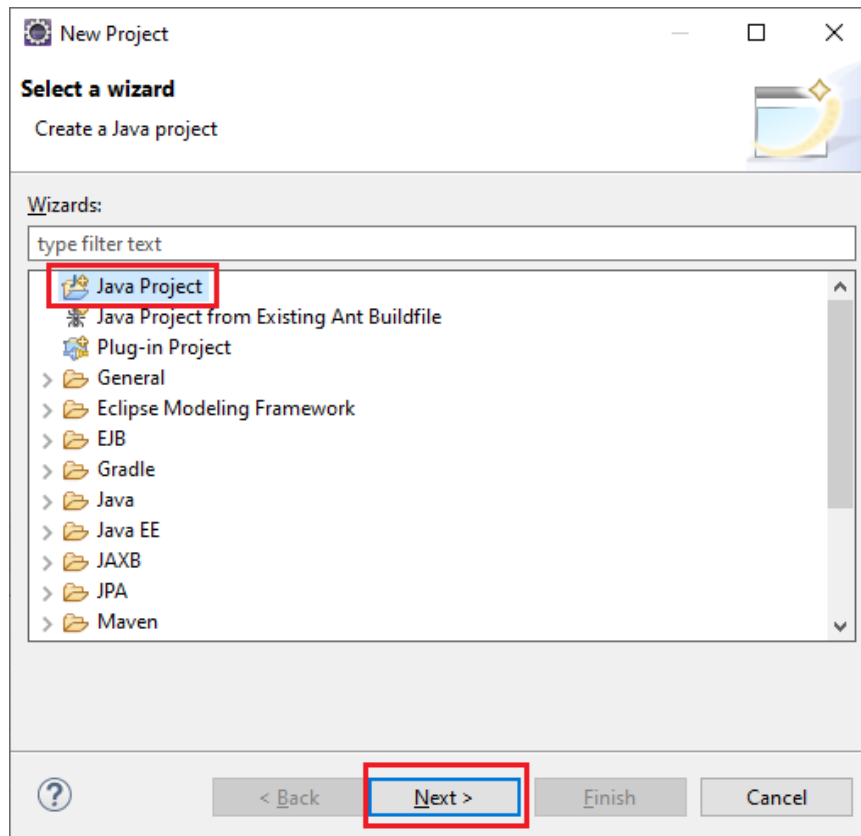


Se invita al alumno a probar diferentes combinaciones, editando y añadiendo usuarios para familiarizarse con el uso (si se indica un usuario con id = 0 se creará un nuevo usuario).

4. Consumir el servicio desde un cliente

Como ya se ha comentado, el propósito de los servicios web es permitir que sean consumidos por otras aplicaciones. A continuación vamos a crear un cliente JAVA que permita consumir el servicio web REST.

En este caso vamos a crear una aplicación de consola, no una aplicación web, por tanto crearemos un proyecto de tipo *Java Project* llamado *EjemploRestAPI_Client*:



En la siguiente pantalla, es importante desmarcar la casilla “Create module-info.java file”, ya que si se deja marcada nos causará problemas al ejecutar:

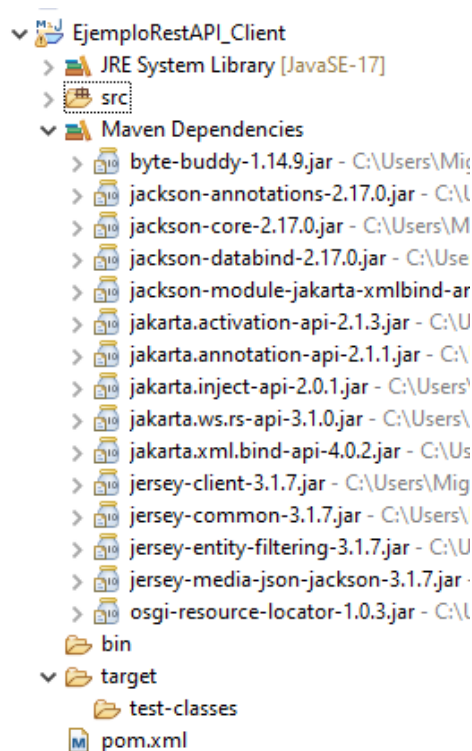
The screenshot shows the 'New Java Project' dialog in Eclipse. The 'Project name' field contains 'EjemploRestAPI_Client'. The 'Location' field shows the path 'C:\Users\Miguel\workspace\EjemploRestAPI_Client'. The 'JRE' section has 'Use an execution environment JRE' selected with 'JavaSE-17'. The 'Project layout' section has 'Create separate folders for sources and class files' selected. The 'Working sets' section has 'Add project to working sets' unchecked. The 'Module' section has 'Create module-info.java file' unchecked. The 'Finish' button is highlighted with a red box.

A continuación lo convertiremos en un proyecto maven, tal y como hicimos para el proyecto Server, y añadiremos las siguientes dependencias en el pom.xml (ojo, no son las mismas que para el servidor).

```
<dependencies>
  <dependency>
    <groupId>org.glassfish.jersey.core</groupId>
    <artifactId>jersey-client</artifactId>
    <version>3.1.7</version>
  </dependency>
  <dependency>
    <groupId>org.glassfish.jersey.media</groupId>
    <artifactId>jersey-media-json-jackson</artifactId>
    <version>3.1.7</version>
  </dependency>
</dependencies>
```

</dependencies>

Es decir, utilizamos el mismo framework (*jersey*) pero para el cliente no son necesarias tantas librerías.



A continuación crearemos un paquete *restclient.data*, y en él una clase Ciudadano que será exactamente igual a la del proyecto servidor (sólo cambia el paquete):


```

Ciudadano.java X
1 package restclient.data;
2
3 public class Ciudadano {
4     private int id;
5
6     private String nombre;
7
8     private String apellidos;
9
10    public int getId() {
11        return id;
12    }
13
14    public void setId(int id) {
15        this.id = id;
16    }
17
18    public String getNombre() {
19        return nombre;
20    }
21
22    public void setNombre(String nombre) {
23        this.nombre = nombre;
24    }
25
26    public String getApellidos() {
27        return apellidos;
28    }
29
30    public void setApellidos(String apellidos) {
31        this.apellidos = apellidos;
32    }
33 }
34

```

Después crearemos un paquete *restclient.api* y en él una clase que nos servirá de pasarela para invocar los métodos del api:

```

package restclient.api;

import java.util.List;
import jakarta.ws.rs.client.Client;
import jakarta.ws.rs.client.ClientBuilder;
import jakarta.ws.rs.client.Entity;
import jakarta.ws.rs.client.Invocation;
import jakarta.ws.rs.client.WebTarget;
import jakarta.ws.rs.core.GenericType;
import jakarta.ws.rs.core.MediaType;
import jakarta.ws.rs.core.Response;
import restclient.data.Ciudadano;

/**
 * Clase que sirve para invocar los métodos del API remoto.
 */
public class ConsumidorCiudadanoAPI {

    private WebTarget webTarget;

    public ConsumidorCiudadanoAPI() {
        Client client = ClientBuilder.newClient();

        // creamos un target para la raíz del api
        webTarget = client.target("http://localhost:8080/EjemploRestAPI_Server/api/ciudadano");
    }
}

```

```

/**
 * Obtiene todos los ciudadanos.
 *
 * @return
 */
public List<Ciudadano> get() {
    //creamos un invocation builder de tipo JSON:
    Invocation.Builder invocationBuilder = webTarget.request(MediaType.APPLICATION_JSON);

    //invocamos get, indicando como argumento que esperamos recibir una lista de ciudadanos:
    return invocationBuilder.get(new GenericType<List<Ciudadano>>() {
    });
}

/**
 * Obtiene el ciudadano con el id indicado
 *
 * @param id el id del ciudadano a obtener
 *
 * @return
 */
public Ciudadano get(int id) {
    // necesitamos un target con el path que incluya el id,
    // lo construimos concatenando el id al target original:
    WebTarget targetObtenerCiudadano = webTarget.path(String.valueOf(id));

    //creamos un invocation builder de tipo JSON:
    Invocation.Builder invocationBuilder =
        targetObtenerCiudadano.request(MediaType.APPLICATION_JSON);

    //invocamos get, indicando como argumento que esperamos recibir un ciudadano:
    return invocationBuilder.get(Ciudadano.class);
}

/**
 * Inserta o actualiza un ciudadano. Si el id indicado es cero,
 * se inserta un nuevo ciudadano, en caso contrario se actualiza
 * el ciudadano con el id indicado.
 * @param ciudadano
 * @return true si la operación se realiza correctamente.
 */
public boolean edit(Ciudadano ciudadano)
{
    //creamos invocation builder de tipo JSON:
    Invocation.Builder invocationBuilder = webTarget.request(MediaType.APPLICATION_JSON);

    //invocamos post, en este caso pasando como argumento el ciudadano modificado:
    Response response =
        invocationBuilder.post(Entity.entity(ciudadano, MediaType.APPLICATION_JSON));

    //la respuesta será correcta si el código devuelto es 200:
    return response.getStatus() == 200;
}

```

```
}
```

Finalmente crearemos una clase ejecutable en el paquete *restclient* para probar el cliente:

```
package restclient;

import java.util.List;

import restclient.api.ConsumidorCiudadanoAPI;
import restclient.data.Ciudadano;

public class RestClientTestApp {
    public static void main(String[] args) {

        ConsumidorCiudadanoAPI ciudadanoAPI = new ConsumidorCiudadanoAPI();

        // Obtener un ciudadano:
        Ciudadano ciudadano1 = ciudadanoAPI.get(1);
        System.out.println("=====");
        System.out.println(" Obtener un ciudadano:");
        mostrarCiudadano(ciudadano1);

        // Obtener todos los ciudadanos:
        System.out.println("=====");
        System.out.println(" Obtener todos los ciudadanos:");
        List<Ciudadano> ciudadanos = ciudadanoAPI.get();
        ciudadanos.stream().forEach(c -> mostrarCiudadano(c));

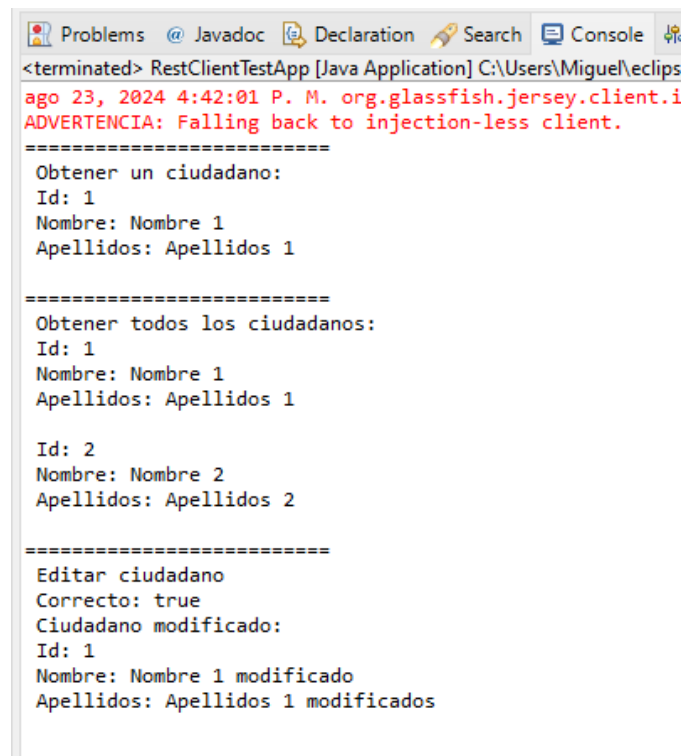
        // editar un ciudadano (post):
        ciudadano1.setNombre("Nombre 1 modificado");
        ciudadano1.setApellidos("Apellidos 1 modificados");
        boolean correcto = ciudadanoAPI.edit(ciudadano1);

        System.out.println("=====");
        System.out.println(" Editar ciudadano");
        System.out.println(" Correcto: " + correcto);

        // recuperamos el ciudadano de nuevo a ver si se ha modificado:
        ciudadano1 = ciudadanoAPI.get(1);
        System.out.println(" Ciudadano modificado:");
        mostrarCiudadano(ciudadano1);
    }

    public static void mostrarCiudadano(Ciudadano ciudadano) {
        System.out.println(" Id: " + ciudadano.getId());
        System.out.println(" Nombre: " + ciudadano.getNombre());
        System.out.println(" Apellidos: " + ciudadano.getApellidos());
        System.out.println();
    }
}
```

Si ejecutamos dicha clase como una aplicación Java, podremos observar cómo nuestro consumidor del servicio se comunica con el servicio web desplegado en local (lógicamente es necesario tener arrancado el servicio desarrollado en el proyecto EjemploRestAPI_Server):



```
Problems @ Javadoc Declaration Search Console
<terminated> RestClientTestApp [Java Application] C:\Users\Miguel\eclips
ago 23, 2024 4:42:01 P. M. org.glassfish.jersey.client.i
ADVERTENCIA: Falling back to injection-less client.
=====
Obtener un ciudadano:
Id: 1
Nombre: Nombre 1
Apellidos: Apellidos 1
=====
Obtener todos los ciudadanos:
Id: 1
Nombre: Nombre 1
Apellidos: Apellidos 1

Id: 2
Nombre: Nombre 2
Apellidos: Apellidos 2
=====
Editar ciudadano
Correcto: true
Ciudadano modificado:
Id: 1
Nombre: Nombre 1 modificado
Apellidos: Apellidos 1 modificados
```

En este caso hemos hecho un cliente en una aplicación de consola, pero se podría implementar de manera análoga en otros tipos de aplicaciones, como aplicaciones web o aplicaciones de escritorio, según nuestras necesidades.